Manuel Complet — React Router

Guide détaillé : de la théorie à l'atelier pratique

Support de cours

16 juillet 2026

Table des matières

1 Introduction générale 4

2 Chapitre 1 — Routeur React : Vue d'ensemble 5
- 2.1 1.1 Le problème que React Router résout . . . 5
- 2.2 1.2 Historique et versions . . . 5
- 2.3 1.3 Installation . . . 5
- 2.4 1.4 Les trois grandes familles d'API . . . 5
- 2.5 1.5 Schéma général d'une application routée . . . 6

3 Chapitre 2 — Principes de base de React Router 7
- 3.1 2.1 Les composants indispensables . . . 7
- 3.2 2.2 Routes dynamiques (paramètres d'URL) . . . 8
- 3.3 2.3 Route « catch-all » (404) . . . 8
- 3.4 2.4 Récapitulatif des concepts clés . . . 8

4 Chapitre 3 — History, Links and Routes 9
- 4.1 3.1 L'objet `history` et l'API History du navigateur . . . 9
- 4.2 3.2 `location` : représenter l'URL actuelle . . . 9
- 4.3 3.3 Différence entre Link, NavLink et navigation programmatique . . . 9
- 4.4 3.4 Anatomie d'une Route . . . 10
- 4.5 3.5 Schéma récapitulatif . . . 10

5 Chapitre 4 — Démo 1 : Routage de base 11
- 5.1 4.1 Objectif de la démo . . . 11
- 5.2 4.2 Structure des fichiers . . . 11
- 5.3 4.3 Point d'entrée : `main.jsx` . . . 11
- 5.4 4.4 Les pages . . . 11
- 5.5 4.5 Le composant `App.jsx` : navigation + routes . . . 12
- 5.6 4.6 Points d'attention pédagogiques . . . 12
- 5.7 4.7 Résultat attendu . . . 13

6 Chapitre 5 — Router Hooks 14
- 6.1 5.1 `useParams()` . . . 14
- 6.2 5.2 `useNavigate()` . . . 14

6.3 5.3 `useLocation()` . . . 14
6.4 5.4 `useSearchParams()` . . . 15
6.5 5.5 `useOutletContext()` . . . 15
6.6 5.6 `useMatch()` . . . 15
6.7 5.7 Tableau récapitulatif . . . 15

7 Chapitre 6 — The Nested Routes (Routes imbriquées) 17
7.1 6.1 Pourquoi imbriquer des routes ? . . . 17
7.2 6.2 Déclaration d'une hiérarchie de routes . . . 17
7.3 6.3 La route `index` . . . 17
7.4 6.4 Chemins relatifs vs absolus . . . 17
7.5 6.5 Ce qu'il manque encore : afficher les enfants . . . 17

8 Chapitre 7 — Nested Routes with Outlet 18
8.1 7.1 Le composant `Outlet` . . . 18
8.2 7.2 Schéma visuel . . . 18
8.3 7.3 Transmettre des données au travers de `Outlet` . . . 18
8.4 7.4 Routes imbriquées à plusieurs niveaux . . . 19

9 Chapitre 8 — Routes privées (Protected Routes) 20
9.1 8.1 Objectif . . . 20
9.2 8.2 Principe général . . . 20
9.3 8.3 Implémentation avec `Outlet` (pour un groupe de routes) . . . 20
9.4 8.4 Implémentation "wrapper" (pour une seule route) . . . 21
9.5 8.5 Mémoriser la page demandée (redirection après connexion) . . . 21
9.6 8.6 Protection par rôle . . . 21
9.7 8.7 Bonnes pratiques . . . 22

10 Chapitre 9 — Conclusion 23
10.1 9.1 Synthèse des concepts vus . . . 23
10.2 9.2 Ce qu'il faut retenir absolument . . . 23
10.3 9.3 Pour aller plus loin . . . 23

11 Chapitre 10 — React Router Checkpoint 24
11.1 Question 1 . . . 24
11.2 Question 2 . . . 24
11.3 Question 3 . . . 24
11.4 Question 4 . . . 24
11.5 Question 5 . . . 24
11.6 Question 6 . . . 24
11.7 Question 7 . . . 25
11.8 Question 8 (mise en situation) . . . 25

12 Chapitre 11 — React Router : One To One 26
12.1 11.1 Questions de compréhension conceptuelle . . . 26
12.2 11.2 Questions de mise en pratique . . . 26
12.3 11.3 Pièges fréquents à discuter . . . 26
12.4 11.4 Grille d'auto-évaluation . . . 26

13 Chapitre 12 — Atelier React Router **28**
13.1 12.1 Contexte du projet 28
13.2 12.2 Cahier des charges technique 28
13.3 12.3 Arborescence suggérée 28
13.4 12.4 Extrait de correction — `AuthContext.jsx` 29
13.5 12.5 Extrait de correction — `App.jsx` 29
13.6 12.6 Extrait de correction — `LayoutEspaceMembre.jsx` 30
13.7 12.7 Critères de validation de l'atelier 31
13.8 12.8 Pour prolonger l'atelier 31

14 Annexe — Aide-mémoire (Cheat Sheet) **32**

1 Introduction générale

React est une bibliothèque conçue pour construire des interfaces utilisateur, mais elle ne fournit **nativement aucun système de navigation entre plusieurs “pages”**. Une application React classique est une **Single Page Application (SPA)** : une seule page HTML est chargée par le navigateur, et c’est le JavaScript qui se charge d’afficher dynamiquement des composants différents selon l’URL.

React Router est la bibliothèque de référence qui comble ce manque. Elle permet de :

- synchroniser l’interface affichée avec l’URL du navigateur ;
- naviguer sans recharger la page (donc sans perdre l’état de l’application) ;
- gérer des paramètres dynamiques dans l’URL (ex. `/produits/12`) ;
- organiser des routes imbriquées (layouts partagés) ;
- protéger certaines routes (authentification) ;
- utiliser l’historique du navigateur (bouton précédent/suivant).

Ce manuel couvre, chapitre par chapitre, l’ensemble du programme : vue d’ensemble, principes de base, historique/liens/routes, démo de routage, hooks, routes imbriquées, `Outlet`, routes privées, un checkpoint, une séance one-to-one et un atelier pratique.

2 Chapitre 1 — Routeur React : Vue d'ensemble

2.1 1.1 Le problème que React Router résout

Dans un site "classique" (non-SPA), chaque clic sur un lien envoie une nouvelle requête HTTP au serveur, qui renvoie une page HTML complète. Cela fonctionne mais :

- provoque un **rechargement complet** de la page (flash blanc, perte de l'état) ;
- est plus lent (tout le HTML/CSS/JS est retéléchargé et réinterprété) ;
- casse l'expérience "application" que l'on veut donner à un SPA.

React Router intercepte les clics sur les liens et met à jour l'URL **via l'API History du navigateur**, sans recharger la page. React se contente de re-rendre les composants correspondant à la nouvelle URL.

2.2 1.2 Historique et versions

React Router a connu plusieurs générations d'API :

Version	Particularité principale
v3	Configuration centralisée des routes en objets
v4 / v5	Approche "déclarative" avec `<Route>` en JSX, imbrication libre
v6	`Routes`/`Route` repensés, `element` au lieu de `component`, hooks (`useNavigate`, `useParams`…), `Outlet` pour les routes imbriquées
v6.4+ / v7	Data APIs (`loader`, `action`, `createBrowserRouter`), gestion des erreurs, chargement de données avant rendu

Ce manuel se concentre sur les concepts de **React Router v6**, qui restent valables (avec un vocabulaire quasi identique) dans les versions plus récentes.

2.3 1.3 Installation

```
npm install react-router-dom
```

`react-router-dom` est le package destiné au **web** (navigateur). Il existe aussi `react-router-native` pour React Native.

2.4 1.4 Les trois grandes familles d'API

1. **Composants** : `<BrowserRouter>`, `<Routes>`, `<Route>`, `<Link>`, `<NavLink>`, `<Outlet>`, `<Navigate>`.

2. **Hooks** : `useNavigate`, `useParams`, `useLocation`, `useSearchParams`, `useOutletContext`, `useNavigation`.
3. **Fonctions utilitaires** (API “data router”, plus avancée) : `createBrowserRouter`, `loader`, `action`, `useLoaderData`.

2.5 1.5 Schéma général d’une application routée

```
<BrowserRouter>             <-- fournit le contexte de routage (History API)
  <Routes>                   <-- choisit UNE route qui correspond à l'URL
    <Route path="/" element={<Accueil/>} />
    <Route path="/contact" element={<Contact/>} />
    <Route path="*" element={<NotFound/>} />   <-- route "catch-all"
  </Routes>
</BrowserRouter>
```

À retenir : `<Routes>` est un **switch intelligent** — il ne rend que le premier `<Route>` dont le `path` correspond à l’URL courante (l’ordre de spécificité est géré automatiquement en v6, contrairement à la v5 où l’ordre déclaratif comptait).

3 Chapitre 2 — Principes de base de React Router

3.1 2.1 Les composants indispensables

3.1.1 `BrowserRouter`

Composant racine qui doit **envelopper toute l'application** (généralement dans `index.js`/`main.jsx`). Il utilise l'API `history` du navigateur (`pushState`/`popState`) pour manipuler l'URL sans rechargement.

```
import { BrowserRouter } from "react-router-dom";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

Il existe des variantes :

- `HashRouter` : utilise `#` dans l'URL (`/#/contact`), utile pour l'hébergement statique sans configuration serveur.
- `MemoryRouter` : garde l'historique en mémoire (utile pour les tests, ou React Native).

3.1.2 `Routes` et `Route`

```
import { Routes, Route } from "react-router-dom";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Accueil />} />
      <Route path="/apropos" element={<APropos />} />
      <Route path="/produits" element={<Produits />} />
    </Routes>
  );
}
```

- `path` : le motif d'URL à matcher.
- `element` : le composant JSX à afficher (v6 utilise `element`, contrairement à `component`/`render` en v5).

3.1.3 `Link`

Remplace la balise `<a>` classique. Un `<a href>` provoquerait un rechargement complet ; `<Link to>` intercepte le clic et met à jour l'historique en JS pur.

```
import { Link } from "react-router-dom";

<Link to="/produits">Voir les produits</Link>
```

3.1.4 `NavLink`

Identique à `Link`, mais ajoute automatiquement une classe CSS (`active` par défaut) quand le lien correspond à l'URL courante — pratique pour les menus de navigation.

```
<NavLink
  to="/produits"
  className={({ isActive }) => (isActive ? "lien-actif" : "lien")}
>
  Produits
</NavLink>
```

3.2 2.2 Routes dynamiques (paramètres d'URL)

```
<Route path="/produits/:id" element={<DetailProduit />} />
```

Le segment `:id` est un **paramètre dynamique**, récupérable dans le composant via le hook `useParams` (voir Chapitre 5).

3.3 2.3 Route « catch-all » (404)

```
<Route path="*" element={<PageNonTrouvee />} />
```

Cette route doit être placée **en dernier** : elle capture toute URL qui n'a matché aucune route précédente.

3.4 2.4 Récapitulatif des concepts clés

Notion	Rôle
`BrowserRouter`	Active le routage basé sur l'historique du navigateur
`Routes`	Sélectionne la meilleure route correspondant à l'URL
`Route`	Associe un `path` à un composant (`element`)
`Link` / `NavLink`	Navigue sans rechargement de page
`:param`	Segment dynamique dans l'URL
`path="*"`	Route de secours (404)

4 Chapitre 3 — History, Links and Routes

4.1 3.1 L'objet `history` et l'API History du navigateur

Avant React Router, il faut comprendre ce sur quoi il repose : l'**API History** native de JavaScript, disponible via `window.history`.

- `history.pushState(state, title, url)` : ajoute une nouvelle entrée dans l'historique **sans recharger la page**.
- `history.replaceState(state, title, url)` : remplace l'entrée courante (pas de nouvelle entrée créée).
- `history.back()` / `history.forward()` / `history.go(n)` : navigue dans l'historique.
- L'évènement `popstate` se déclenche quand l'utilisateur clique sur les boutons précédent/suivant du navigateur.

React Router encapsule cette API dans un objet interne (`history`) que ses composants et hooks utilisent. C'est ce qui permet à `Link`, `NavLink` et `useNavigate` de changer l'URL de façon fluide, tout en gardant la pile de navigation du navigateur cohérente (les boutons précédent/suivant du navigateur continuent de fonctionner correctement).

4.2 3.2 `location` : représenter l'URL actuelle

Chaque fois que l'URL change, React Router expose un objet `location` avec la structure suivante :

```
{
  pathname: "/produits/12",
  search: "?tri=prix",
  hash: "#avis",
  state: { depuisPage: "accueil" },  // état "invisible" transmis lors de la navigation
  key: "ac3df9"
}
```

- `pathname` : le chemin de l'URL.
- `search` : la chaîne de requête (query string).
- `hash` : l'ancre (`#section`).
- `state` : donnée transmise par programme (via `navigate(path, { state })`), invisible dans l'URL.

4.3 3.3 Différence entre Link, NavLink et navigation programmatique

Méthode	Cas d'usage
`<Link to="...">`	Navigation déclarative simple, dans un rendu JSX
`<NavLink to="...">`	Comme `Link`, mais avec état "actif" (menus)

Méthode	Cas d'usage
`useNavigate()`	Navigation **programmatique** (après une soumission de formulaire, un `setTimeout`, une condition métier…)
`<Navigate to="...">`	Redirection déclarative dans le rendu (ex. route protégée)

4.4 3.4 Anatomie d'une Route

```
<Route
  path="/produits/:id"
  element={<DetailProduit />}
/>
```

Quand l'URL matche `path`, React Router :

1. calcule les **paramètres** (`:id` → valeur réelle) ;
2. rend le composant indiqué dans `element` ;
3. rend disponibles, via les hooks, `location`, `params`, `searchParams` à l'intérieur de ce composant et de ses enfants.

4.5 3.5 Schéma récapitulatif

```
Clic utilisateur sur <Link to="/produits/12">
        |
        ▼
history.pushState(...)  (géré en interne par React Router)
        |
        ▼
L'URL du navigateur change SANS rechargement
        |
        ▼
<Routes> recalcule quelle <Route> correspond à /produits/12
        |
        ▼
Le composant <DetailProduit/> est monté / re-rendu
```

5 Chapitre 4 — Démo 1 : Routage de base

5.1 4.1 Objectif de la démo

Construire une mini-application avec 3 pages (Accueil, À propos, Contact) et un menu de navigation, afin de mettre en pratique `BrowserRouter`, `Routes`, `Route`, `Link` et `NavLink`.

5.2 4.2 Structure des fichiers

```
src/
 ├── App.jsx
 ├── main.jsx
 └── pages/
     ├── Accueil.jsx
     ├── APropos.jsx
     ├── Contact.jsx
     └── PageNonTrouvee.jsx
```

5.3 4.3 Point d'entrée : `main.jsx`

```jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

5.4 4.4 Les pages

```jsx
// pages/Accueil.jsx
export default function Accueil() {
  return <h1>Bienvenue sur la page d'accueil</h1>;
}
```

```jsx
// pages/APropos.jsx
export default function APropos() {
  return <h1>À propos de nous</h1>;
}
```

```jsx
// pages/Contact.jsx
export default function Contact() {
  return <h1>Contactez-nous</h1>;
}
```

```jsx
// pages/PageNonTrouvee.jsx
export default function PageNonTrouvee() {
  return <h1>404 – Page introuvable</h1>;
}
```

5.5 4.5 Le composant `App.jsx` : navigation + routes

```jsx
import { Routes, Route, NavLink } from "react-router-dom";
import Accueil from "./pages/Accueil";
import APropos from "./pages/APropos";
import Contact from "./pages/Contact";
import PageNonTrouvee from "./pages/PageNonTrouvee";

export default function App() {
  return (
    <div>
      <nav>
        <NavLink to="/" end>Accueil</NavLink>
        <NavLink to="/apropos">À propos</NavLink>
        <NavLink to="/contact">Contact</NavLink>
      </nav>

      <Routes>
        <Route path="/" element={<Accueil />} />
        <Route path="/apropos" element={<APropos />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<PageNonTrouvee />} />
      </Routes>
    </div>
  );
}
```

5.6 4.6 Points d'attention pédagogiques

- L'attribut `end` sur `<NavLink to="/" end>` est **essentiel** : sans lui, le lien "Accueil" resterait actif sur toutes les routes, car `/` est un préfixe de toutes les URLs.
- L'ordre des `<Route>` n'a pas d'importance en v6 pour les chemins statiques (React Router calcule la meilleure correspondance), **sauf** pour la route `*` qui doit toujours être en dernier par convention de lisibilité.
- Chaque page est un composant isolé : c'est une bonne pratique de séparer chaque "vue" dans son propre fichier.

5.7 4.7 Résultat attendu

En lançant `npm run dev`, cliquer sur les liens du menu doit changer le contenu affiché **sans rechargement du navigateur** (vérifiable en observant qu'aucun "flash blanc" n'apparaît, et que l'URL change dans la barre d'adresse).

6 Chapitre 5 — Router Hooks

Les hooks de React Router permettent d'accéder, **depuis n'importe quel composant rendu à l'intérieur d'un `Router`**, aux informations de navigation sans avoir à les faire transiter par des props.

6.1 5.1 `useParams()`

Récupère les paramètres dynamiques définis dans le `path` de la route.

```
// Route: <Route path="/produits/:id" element={<DetailProduit/>} />

import { useParams } from "react-router-dom";

function DetailProduit() {
  const { id } = useParams();
  return <p>Produit numéro {id}</p>;
}
```

6.2 5.2 `useNavigate()`

Permet une navigation **programmatique** (déclenchée par du code, pas par un clic direct sur un lien).

```
import { useNavigate } from "react-router-dom";

function FormulaireConnexion() {
  const navigate = useNavigate();

  function handleSubmit(e) {
    e.preventDefault();
    // ... logique de connexion
    navigate("/tableau-de-bord");        // navigation simple
    // navigate(-1);                     // retour en arrière (comme le bouton précéden
    // navigate("/profil", { replace: true }); // remplace l'entrée d'historique
  }

  return <form onSubmit={handleSubmit}>{/* ... */}</form>;
}
```

6.3 5.3 `useLocation()`

Renvoie l'objet `location` courant (pathname, search, hash, state).

```
import { useLocation } from "react-router-dom";

function Analytics() {
  const location = useLocation();
```

```
  useEffect(() => {
    console.log("Nouvelle page vue :", location.pathname);
  }, [location]);

  return null;
}
```

6.4 5.4 useSearchParams()

Gère la **query string** (`?tri=prix&page=2`), à la manière d'un `useState` dédié aux paramètres d'URL.

```
import { useSearchParams } from "react-router-dom";

function ListeProduits() {
  const [searchParams, setSearchParams] = useSearchParams();
  const tri = searchParams.get("tri") || "nom";

  function changerTri(nouveauTri) {
    setSearchParams({ tri: nouveauTri });
  }

  return (
    <div>
      <button onClick={() => changerTri("prix")}>Trier par prix</button>
      <p>Tri actuel : {tri}</p>
    </div>
  );
}
```

6.5 5.5 useOutletContext()

Permet de transmettre des données d'un layout parent (qui contient `<Outlet/>`) vers les routes enfants, sans passer par le Context API "manuel" (voir Chapitre 7).

6.6 5.6 useMatch()

Vérifie si un chemin donné correspond à l'URL actuelle (utile pour de la logique conditionnelle fine, en dehors des styles de `NavLink`).

6.7 5.7 Tableau récapitulatif

Hook	Retourne	Usage typique
`useParams()`	Objet des paramètres dynamiques `{id: "12"}`	Récupérer un identifiant dans l'URL

Hook	Retourne	Usage typique
`useNavigate()`	Fonction `navigate(to, options)`	Redirections après action (formulaire, timer, condition)
`useLocation()`	Objet `location` courant	Suivi analytique, affichage conditionnel selon la page
`useSearchParams()`	`[searchParams, setSearchParams]`	Filtres, tri, pagination via l'URL
`useOutletContext()`	Contexte transmis par le parent	Partager des données entre layout et routes enfants
`useMatch(pattern)`	Résultat de correspondance ou `null`	Logique conditionnelle sur l'URL

7 Chapitre 6 — The Nested Routes (Routes imbriquées)

7.1 6.1 Pourquoi imbriquer des routes ?

De nombreuses applications partagent une structure visuelle commune entre plusieurs pages : un en-tête, un menu latéral, un pied de page. Sans routes imbriquées, il faudrait dupliquer ce layout dans chaque composant de page.

Les **routes imbriquées** permettent de définir un **layout parent une seule fois**, et de n'échanger que la partie centrale du contenu selon l'URL.

7.2 6.2 Déclaration d'une hiérarchie de routes

```
<Routes>
  <Route path="/tableau-de-bord" element={<LayoutTableauDeBord />}>
    <Route index element={<Accueil />} />
    <Route path="statistiques" element={<Statistiques />} />
    <Route path="parametres" element={<Parametres />} />
  </Route>
</Routes>
```

- `/tableau-de-bord` affiche `LayoutTableauDeBord` avec `Accueil` à l'intérieur (grâce à `index`).
- `/tableau-de-bord/statistiques` affiche `LayoutTableauDeBord` avec `Statistiques` à l'intérieur.
- `/tableau-de-bord/parametres` affiche `LayoutTableauDeBord` avec `Parametres` à l'intérieur.

7.3 6.3 La route `index`

`index` désigne la route **par défaut** affichée quand l'URL correspond exactement au chemin du parent, sans segment supplémentaire. Elle ne prend pas de `path`.

7.4 6.4 Chemins relatifs vs absolus

Dans une route imbriquée, `path="statistiques"` (sans `/` au début) est **relatif** au chemin du parent — il devient donc `/tableau-de-bord/statistiques`. Cela évite de répéter le préfixe à chaque niveau.

7.5 6.5 Ce qu'il manque encore : afficher les enfants

Déclarer la hiérarchie ne suffit pas : il faut indiquer, **dans le composant parent**, *où* le contenu de la route enfant doit s'afficher. C'est le rôle du composant `<Outlet/>`, objet du chapitre suivant.

8 Chapitre 7 — Nested Routes with Outlet

8.1 7.1 Le composant `Outlet`

`<Outlet/>` est un **emplacement réservé** (placeholder) : il indique à React Router où injecter le contenu de la route enfant active, à l'intérieur du composant parent.

```
import { Outlet, NavLink } from "react-router-dom";

function LayoutTableauDeBord() {
  return (
    <div className="dashboard">
      <aside>
        <NavLink to="." end>Accueil</NavLink>
        <NavLink to="statistiques">Statistiques</NavLink>
        <NavLink to="parametres">Paramètres</NavLink>
      </aside>

      <main>
        <Outlet />   {/* <-- le contenu de la route enfant s'affiche ici */}
      </main>
    </div>
  );
}
```

8.2 7.2 Schéma visuel

```
URL: /tableau-de-bord/statistiques

┌──────────────────────────────────────────┐
│  LayoutTableauDeBord (toujours affiché)  │
│  ┌──────────────┐ ┌──────────────────────┐ │
│  │  <aside>     │ │  <main>              │ │
│  │  Menu        │ │   <Outlet/>          │ │
│  │  latéral     │ │   → <Statistiques/>  │ │
│  └──────────────┘ └──────────────────────┘ │
└──────────────────────────────────────────┘
```

Le menu latéral, l'en-tête, etc. **ne se re-montent pas** à chaque navigation interne : seul le contenu de `<Outlet/>` change. C'est un gain de performance et de continuité d'expérience (par exemple, un lecteur audio dans le layout continuerait de jouer).

8.3 7.3 Transmettre des données au travers de `Outlet`

```
// Layout parent
<Outlet context={{ utilisateur }} />
```

```
// Route enfant
import { useOutletContext } from "react-router-dom";
```

```jsx
function Statistiques() {
  const { utilisateur } = useOutletContext();
  return <p>Bonjour {utilisateur.nom}</p>;
}
```

8.4 7.4 Routes imbriquées à plusieurs niveaux

Rien n'empêche d'imbriquer encore davantage :

```jsx
<Route path="admin" element={<LayoutAdmin />}>
  <Route path="utilisateurs" element={<LayoutUtilisateurs />}>
    <Route index element={<ListeUtilisateurs />} />
    <Route path=":id" element={<DetailUtilisateur />} />
  </Route>
</Route>
```

Chaque niveau de layout doit contenir son propre `<Outlet/>` pour que la chaîne d'affichage fonctionne jusqu'au bout.

9 Chapitre 8 — Routes privées (Protected Routes)

9.1 8.1 Objectif

Empêcher un utilisateur non authentifié (ou non autorisé) d'accéder à certaines pages (ex. `/tableau-de-bord`, `/admin`), en le redirigeant automatiquement vers une page de connexion.

9.2 8.2 Principe général

On crée un composant "garde" (souvent appelé `RouteProtegee` ou `PrivateRoute`) qui :

1. vérifie une condition (utilisateur connecté ? rôle suffisant ?) ;
2. si la condition est vraie → rend `<Outlet/>` (laisse passer vers la route enfant) ;
3. sinon → redirige vers une autre page grâce à `<Navigate/>`.

9.3 8.3 Implémentation avec `Outlet` (pour un groupe de routes)

```jsx
// RouteProtegee.jsx
import { Navigate, Outlet } from "react-router-dom";
import { useAuth } from "./AuthContext";

export default function RouteProtegee() {
  const { utilisateur } = useAuth();

  if (!utilisateur) {
    return <Navigate to="/connexion" replace />;
  }

  return <Outlet />;
}
```

```jsx
// App.jsx
<Routes>
  <Route path="/connexion" element={<Connexion />} />

  <Route element={<RouteProtegee />}>
    <Route path="/tableau-de-bord" element={<TableauDeBord />} />
    <Route path="/profil" element={<Profil />} />
  </Route>
</Routes>
```

Toutes les routes déclarées **en enfants** de `<RouteProtegee/>` sont automatiquement protégées.

9.4 8.4 Implémentation “wrapper” (pour une seule route)

```
function RouteProtegee({ children }) {
  const { utilisateur } = useAuth();
  return utilisateur ? children : <Navigate to="/connexion" replace />;
}

// Utilisation
<Route
  path="/profil"
  element={
    <RouteProtegee>
      <Profil />
    </RouteProtegee>
  }
/>
```

9.5 8.5 Mémoriser la page demandée (redirection après connexion)

```
import { Navigate, Outlet, useLocation } from "react-router-dom";

function RouteProtegee() {
  const { utilisateur } = useAuth();
  const location = useLocation();

  if (!utilisateur) {
    return <Navigate to="/connexion" state={{ from: location }} replace />;
  }
  return <Outlet />;
}
```

```
// Page de connexion : après succès
const location = useLocation();
const navigate = useNavigate();
const destination = location.state?.from?.pathname || "/tableau-de-bord";

function connexionReussie() {
  navigate(destination, { replace: true });
}
```

9.6 8.6 Protection par rôle

```
function RouteProtegeeParRole({ rolesAutorises }) {
  const { utilisateur } = useAuth();

  if (!utilisateur) return <Navigate to="/connexion" replace />;
```

```
  if (!rolesAutorises.includes(utilisateur.role)) {
    return <Navigate to="/non-autorise" replace />;
  }
  return <Outlet />;
}

// Usage
<Route element={<RouteProtegeeParRole rolesAutorises={["admin"]} />}>
  <Route path="/admin" element={<Admin />} />
</Route>
```

9.7 8.7 Bonnes pratiques

- Le `replace` évite d'empiler la page protégée dans l'historique (empêche de revenir dessus avec le bouton précédent après une redirection).
- La vérification d'authentification côté client (React Router) **ne remplace jamais** une vérification côté serveur/API : c'est une protection d'expérience utilisateur, pas une sécurité absolue.
- Prévoir un état de chargement (`isLoading`) si la vérification d'authentification est asynchrone (ex. appel API pour valider un token), afin d'éviter une redirection prématurée vers `/connexion` pendant le chargement.

10 Chapitre 9 — Conclusion

10.1 9.1 Synthèse des concepts vus

Chapitre	Concept clé
Vue d'ensemble	Rôle de React Router dans une SPA
Principes de base	`BrowserRouter`, `Routes`, `Route`, `Link`, `NavLink`
History/Links/Routes	API History, objet `location`, navigation déclarative vs programmatique
Démo 1	Mise en pratique d'un routage simple à 3 pages
Router Hooks	`useParams`, `useNavigate`, `useLocation`, `useSearchParams`, `useOutletContext`
Nested Routes	Layouts partagés, route `index`, chemins relatifs
Outlet	Injection du contenu enfant dans le layout parent
Routes privées	Protection par authentification/rôle, redirection

10.2 9.2 Ce qu'il faut retenir absolument

1. React Router synchronise **l'URL** et **l'interface**, sans jamais recharger la page.
2. `Routes` choisit une seule route par rendu ; `Route` associe chemin et composant.
3. Les hooks (`useParams`, `useNavigate`, etc.) donnent accès à toute l'information de navigation depuis n'importe quel composant enfant.
4. Les routes imbriquées + `Outlet` évitent la duplication de layout et améliorent les performances de rendu.
5. Une route privée n'est qu'un composant conditionnel qui redirige avec `<Navigate/>` si une condition métier n'est pas remplie.

10.3 9.3 Pour aller plus loin

- Les **Data Routers** (`createBrowserRouter`, `loader`, `action`) permettent de charger les données **avant** le rendu du composant, avec gestion d'erreurs intégrée (`errorElement`).
- Le **code splitting** avec `React.lazy + Suspense` combiné à React Router permet de charger le code de chaque page à la demande, pour améliorer les performances.
- La gestion des **transitions animées** entre routes (bibliothèques comme `framer-motion`).

11 Chapitre 10 — React Router Checkpoint

Ce checkpoint est une série d'exercices/questions destinés à valider la compréhension des chapitres précédents. Une proposition de corrigé synthétique est donnée après chaque question.

11.1 Question 1

Quelle est la différence entre `<a href="...">` et `<Link to="...">` ?

Corrigé : `<a href>` provoque une requête HTTP complète et un rechargement de page. `<Link to>` intercepte le clic et utilise l'API History pour changer l'URL **sans rechargement**, en ne re-rendant que les composants React nécessaires.

11.2 Question 2

Comment récupérer un paramètre dynamique `:id` défini dans une route ?

Corrigé : avec le hook `useParams()`, qui renvoie un objet contenant chaque paramètre déclaré dans le `path` (ex. `const { id } = useParams();`).

11.3 Question 3

Pourquoi faut-il ajouter l'attribut `end` sur un `NavLink` pointant vers `/` ?

Corrigé : sans `end`, React Router considère que `/` est un préfixe de toutes les URLs, et le style "actif" resterait appliqué en permanence. `end` force une correspondance exacte.

11.4 Question 4

Quel composant permet de définir un layout partagé entre plusieurs routes ?

Corrigé : on déclare une route parent dont l'`element` est le composant de layout, et on place les routes enfants à l'intérieur ; le layout doit contenir un `<Outlet/>` pour afficher le contenu correspondant à la route enfant active.

11.5 Question 5

Comment rediriger un utilisateur non connecté vers `/connexion` ?

Corrigé : en créant un composant "garde" (route protégée) qui vérifie l'état d'authentification, et qui retourne `<Navigate to="/connexion" replace />` si l'utilisateur n'est pas connecté, ou `<Outlet/>` sinon.

11.6 Question 6

Quelle est la différence entre `useNavigate()` et `<Link>` ?

Corrigé : `<Link>` est déclaratif, utilisé dans le JSX pour un clic utilisateur direct. `useNavigate()` renvoie une fonction utilisable **dans la logique JS** (après une

soumission de formulaire, une condition, un minuteur), pour une navigation programmatique.

11.7 Question 7

À quoi sert `useSearchParams()` ?

Corrigé : à lire et modifier la query string de l'URL (`?tri=prix&page=2`), utile pour des filtres, un tri ou une pagination persistants dans l'URL (partageables/rechargeables).

11.8 Question 8 (mise en situation)

Vous devez créer une route `/blog/:slug` qui affiche un article. Écrivez la déclaration de route et le composant qui affiche le `slug`.

Corrigé :

```
<Route path="/blog/:slug" element={<Article />} />
```

```
function Article() {
  const { slug } = useParams();
  return <h1>Article : {slug}</h1>;
}
```

12 Chapitre 11 — React Router : One To One

Cette section reprend les questions généralement posées lors d'une séance individuelle (one-to-one) avec un formateur, pour vérifier la compréhension en profondeur et discuter des cas limites.

12.1 11.1 Questions de compréhension conceptuelle

1. Pourquoi une SPA a-t-elle besoin d'un routeur côté client alors que le serveur ne renvoie qu'une seule page HTML ?
2. Que se passe-t-il si deux `<Route>` ont un `path` qui pourrait correspondre à la même URL ? Comment React Router choisit-il ?
3. Quelle est la différence entre `replace: true` et un `navigate()` classique du point de vue de l'historique du navigateur ?
4. Pourquoi le `state` transmis via `navigate(path, { state })` n'apparaît-il pas dans l'URL, contrairement à `useSearchParams` ?
5. Dans une route imbriquée, que se passe-t-il si le composant parent **oublie** d'inclure `<Outlet/>` ?

12.2 11.2 Questions de mise en pratique

1. Comment structureriez-vous les routes d'un site e-commerce avec : accueil, catalogue, détail produit, panier, compte utilisateur (protégé), page admin (protégée par rôle) ?
2. Comment géreriez-vous une page 404 personnalisée qui affiche tout de même le menu de navigation du site ?
3. Comment transmettre, depuis un layout parent, les informations de l'utilisateur connecté à toutes les routes enfants, sans prop-drilling ?
4. Comment implémenteriez-vous un fil d'Ariane (breadcrumb) dynamique basé sur les routes imbriquées ?

12.3 11.3 Pièges fréquents à discuter

- **Oublier `BrowserRouter`** à la racine : tous les hooks/composants de routage lèvent une erreur ("useNavigate() may be used only in the context of a component").
- **Confondre chemin absolu et relatif** dans les routes imbriquées (`path="/statistiques"` vs `path="statistiques"`).
- **Utiliser `<a href>` par erreur** dans une SPA, provoquant un rechargement complet non désiré.
- **Ne pas gérer l'état de chargement** dans une route protégée asynchrone, provoquant une redirection prématurée vers la page de connexion avant même que l'authentification n'ait pu être vérifiée.
- **Oublier `replace`** lors d'une redirection post-connexion, ce qui permettrait à l'utilisateur de "revenir" sur une page qui l'a redirigé, créant une boucle de navigation confuse.

12.4 11.4 Grille d'auto-évaluation

Compétence	Maîtrisé	À revoir
Différencier `Link`, `NavLink`, `useNavigate`, `Navigate`		
Créer des routes dynamiques avec paramètres		
Construire un layout avec routes imbriquées et `Outlet`		
Implémenter une route privée avec redirection		
Lire/écrire des paramètres d’URL avec `useSearchParams`		

13 Chapitre 12 — Atelier React Router

13.1 12.1 Contexte du projet

L'atelier consiste à construire une petite application **"MonEspace"** comportant :

- une page d'accueil publique ;
- une page "À propos" publique ;
- une page de connexion ;
- un espace membre protégé, avec un layout comportant un menu latéral et 3 sous-pages : Profil, Statistiques, Paramètres ;
- une page d'administration, accessible uniquement au rôle `admin` ;
- une page 404.

13.2 12.2 Cahier des charges technique

1. Installer `react-router-dom`.
2. Mettre en place `BrowserRouter` à la racine.
3. Créer le menu de navigation public avec `NavLink`.
4. Créer un contexte d'authentification simple (`AuthContext`) avec un état `utilisateur` (nul si non connecté) et une fonction `seConnecter(role)` simulant une connexion.
5. Implémenter la page de connexion : formulaire minimal (email + bouton "Se connecter en tant qu'utilisateur" / "Se connecter en tant qu'admin").
6. Créer le composant `RouteProtegee` (Chapitre 8) et l'utiliser pour envelopper les routes de l'espace membre.
7. Créer le composant `RouteProtegeeParRole` pour la route `/admin`.
8. Construire le layout de l'espace membre avec `<Outlet/>` (Chapitre 7) et une route `index`.
9. Ajouter la route * pour la page 404.
10. Bonus : ajouter `useSearchParams` sur la page Statistiques pour filtrer par période (`?periode=semaine|mois|annee`).

13.3 12.3 Arborescence suggérée

```
src/
 ├── main.jsx
 ├── App.jsx
 ├── context/
 │    └── AuthContext.jsx
 ├── components/
 │    ├── RouteProtegee.jsx
 │    └── RouteProtegeeParRole.jsx
 ├── layouts/
 │    └── LayoutEspaceMembre.jsx
 └── pages/
      ├── Accueil.jsx
      ├── APropos.jsx
      ├── Connexion.jsx
```

```
        ├── Profil.jsx
        ├── Statistiques.jsx
        ├── Parametres.jsx
        ├── Admin.jsx
        └── PageNonTrouvee.jsx
```

13.4 12.4 Extrait de correction — `AuthContext.jsx`

```jsx
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [utilisateur, setUtilisateur] = useState(null);

  function seConnecter(role) {
    setUtilisateur({ nom: "Invité", role });
  }

  function seDeconnecter() {
    setUtilisateur(null);
  }

  return (
    <AuthContext.Provider value={{ utilisateur, seConnecter, seDeconnecter }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}
```

13.5 12.5 Extrait de correction — `App.jsx`

```jsx
import { Routes, Route } from "react-router-dom";
import Accueil from "./pages/Accueil";
import APropos from "./pages/APropos";
import Connexion from "./pages/Connexion";
import Admin from "./pages/Admin";
import PageNonTrouvee from "./pages/PageNonTrouvee";
import LayoutEspaceMembre from "./layouts/LayoutEspaceMembre";
import Profil from "./pages/Profil";
import Statistiques from "./pages/Statistiques";
import Parametres from "./pages/Parametres";
import RouteProtegee from "./components/RouteProtegee";
```

```jsx
import RouteProtegeeParRole from "./components/RouteProtegeeParRole";

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Accueil />} />
      <Route path="/apropos" element={<APropos />} />
      <Route path="/connexion" element={<Connexion />} />

      <Route element={<RouteProtegee />}>
        <Route path="/espace-membre" element={<LayoutEspaceMembre />}>
          <Route index element={<Profil />} />
          <Route path="statistiques" element={<Statistiques />} />
          <Route path="parametres" element={<Parametres />} />
        </Route>
      </Route>

      <Route element={<RouteProtegeeParRole rolesAutorises={["admin"]} />}>
        <Route path="/admin" element={<Admin />} />
      </Route>

      <Route path="*" element={<PageNonTrouvee />} />
    </Routes>
  );
}
```

13.6 12.6 Extrait de correction — `LayoutEspaceMembre.jsx`

```jsx
import { NavLink, Outlet } from "react-router-dom";

export default function LayoutEspaceMembre() {
  return (
    <div className="espace-membre">
      <aside>
        <NavLink to="/espace-membre" end>Profil</NavLink>
        <NavLink to="/espace-membre/statistiques">Statistiques</NavLink>
        <NavLink to="/espace-membre/parametres">Paramètres</NavLink>
      </aside>
      <main>
        <Outlet />
      </main>
    </div>
  );
}
```

13.7 12.7 Critères de validation de l'atelier

- ☐ Naviguer entre Accueil / À propos ne recharge jamais la page.
- ☐ Accéder à `/espace-membre` sans être connecté redirige vers `/connexion`.
- ☐ Après connexion, l'utilisateur est renvoyé vers la page initialement demandée.
- ☐ Le menu latéral de l'espace membre reste affiché lors de la navigation entre Profil/Statistiques/Paramètres (seul `<Outlet/>` change).
- ☐ Un utilisateur connecté avec le rôle `utilisateur` (non-admin) ne peut pas accéder à `/admin`.
- ☐ Une URL invalide affiche la page 404 personnalisée.
- ☐ (Bonus) La page Statistiques lit un filtre de période depuis l'URL via `useSearchParams`.

13.8 12.8 Pour prolonger l'atelier

- Ajouter une animation de transition entre les pages.
- Ajouter un fil d'Ariane dynamique basé sur les segments de l'URL.
- Migrer l'atelier vers l'API `createBrowserRouter` avec des `loader` pour précharger les données de chaque page.

14 Annexe — Aide-mémoire (Cheat Sheet)

```
// Imports courants
import {
  BrowserRouter, Routes, Route,
  Link, NavLink, Navigate, Outlet,
  useParams, useNavigate, useLocation,
  useSearchParams, useOutletContext, useMatch
} from "react-router-dom";
```

Besoin	Solution
Envelopper l'app	`<BrowserRouter>`
Déclarer des pages	`<Routes><Route path="..." element={...}/></Routes>`
Lien cliquable	`<Link to="...">`
Lien avec état actif	`<NavLink to="...">`
Layout partagé	Route parent + `<Outlet/>`
Page par défaut d'un layout	`<Route index element={...}/>`
Paramètre dynamique	`path=":id"` + `useParams()`
Filtre/tri/pagination via URL	`useSearchParams()`
Navigation après action JS	`useNavigate()`
Redirection déclarative	`<Navigate to="..." replace/>`
Route protégée	Composant garde + `<Outlet/>` ou `<Navigate/>`
Page 404	`<Route path="*" element={...}/>`

Fin du manuel.